

SelVerIR: Self-Verifying Programming Language
Intermediate Representation Syntax and Semantics Document
v0.1

Yusuf Demir
Yağız Kaan Aydoğdu

March 6, 2026

1 Syntax

$$\begin{aligned}\langle IR \rangle &::= \{ \langle label \rangle \langle Command \rangle \} \\ \langle Command \rangle &::= \text{DECL } \langle type \rangle, \langle ident \rangle \\ &\quad | \text{PUSH } \langle imm \rangle \\ &\quad | \text{iSTORE } \langle ident \rangle \mid \text{fSTORE } \langle ident \rangle \\ &\quad | \text{LOAD } \langle ident \rangle \\ &\quad | \text{ADD} \\ &\quad | \text{SUB} \\ &\quad | \text{MUL} \\ &\quad | \text{iDIV} \mid \text{fDIV} \\ &\quad | \text{EQ} \\ &\quad | \text{LT} \\ &\quad | \text{LE} \\ &\quad | \text{GT} \\ &\quad | \text{GE} \\ &\quad | \text{NEG} \\ &\quad | \text{AND} \\ &\quad | \text{OR} \\ &\quad | \text{XOR} \\ &\quad | \text{JZ } \langle int \rangle \\ &\quad | \text{GOTO } \langle int \rangle \\ &\quad | \text{NOOP} \\ \\ \langle type \rangle &::= \text{INT} \mid \text{FLOAT} \mid \text{LIST}[\langle type \rangle, \langle int \rangle] \\ \langle ident \rangle &::= [\text{A-Za-z}][\text{A-Za-z0-9}]^* \\ \langle label \rangle &::= [0-9]^+ : \\ \langle int \rangle &::= [0-9]^+ \\ \langle float \rangle &::= [0-9]^+ . [0-9]^+ \\ \langle imm \rangle &::= \langle int \rangle \mid \langle float \rangle\end{aligned}$$

2 Semantics

2.1 Mathematical notions

The **SelVeri** intermediate representation, **SelVerIR** is executed on a *stack-based* abstract machine (**AM**) and aims to be provably correct implementation of the high-level **SelVeri** language.

2.1.1 Stack

At any moment of the execution of a **SelVerIR** program, its stack, denoted as e , is a an ordered finite sequence consisting elements are from **Val**, which will be used in the flow of the program in the *LIFO* order. As expected, e supports **PUSH** and (indirectly) **POP** operations. For convenience, the empty stack is denoted separately, as ϵ .

2.1.2 Program counter

Intuitively, one can think of any **SelVerIR** program, P as an ordered list of commands. Program counter, pc , represents the index of the command that is to be executed next. Control-flow statements such as **GOTO** and **JZ** can alter the program counter in a non-sequential manner.

2.1.3 IR-configuration

An IR-configuration is the representation of the execution state in the form of a 5-tuple $\langle c, e, \sigma, s, pc \rangle$ where

- c is the list of commands representing the remaining part of the program **SelVerIR** program, i.e, $c = P[pc:]$.
- e is the stack of the execution state,
- σ is the program state,
- s is the scope,
- pc is the program counter.

2.2 Small-step operational semantics of SelVerIR commands

Let P be any SelVerIR program throughout the section 2.2.

2.2.1 Small-step operational semantics of state-related commands

$\langle \text{DECL } t : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma[x \mapsto \mathbf{0}], s[x \mapsto t], pc + 1 \rangle \\ \text{if } x \in \mathbf{Var} \wedge t \in \mathbf{Type} \wedge s(x) = \mathbf{Void} \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{iSTORE } x : c, z : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma[x \mapsto z], s, pc + 1 \rangle & \text{if } s(x) = \mathbf{Int} \wedge z \in \mathbb{Z} \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{fSTORE } x : c, r : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma[x \mapsto r], s, pc + 1 \rangle & \text{if } s(x) = \mathbf{Float} \wedge r \in \mathbb{R} \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{LOAD } x : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, \sigma(x, s) : e, \sigma, s, pc + 1 \rangle & \text{if } x \in \mathbf{Var} \wedge \sigma(x, s) \neq \perp \\ \langle c, e, \sigma, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{PUSH } v : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, \mathcal{V}(v) : e, \sigma, s, pc + 1 \rangle \quad \text{if } v \in \mathbf{Imm}$

Table 1: Operational semantics of state-related commands

2.2.2 Small-step operational semantics of arithmetic commands

$\langle \text{ADD} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 + v_2) : e, \sigma, s, pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \text{SUB} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 - v_2) : e, \sigma, s, pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \text{MUL} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 \cdot v_2) : e, \sigma, s, pc + 1 \rangle \quad \text{if } v_1, v_2 \in \mathbb{R}$
$\langle \text{iDIV} : c, z_1 : z_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, \lfloor z_1 / z_2 \rfloor : e, \sigma, s, pc + 1 \rangle & \text{if } z_1, z_2 \in \mathbb{Z} \wedge z_2 \neq 0 \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$
$\langle \text{fDIV} : c, r_1 : r_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, r_1 / r_2 : e, \sigma, s, pc + 1 \rangle & \text{if } r_1, r_2 \in \mathbb{R} \wedge r_2 \neq 0 \\ \langle c, e, \hat{\sigma}, s, pc + 1 \rangle & \text{otherwise} \end{cases}$

Table 2: Operational semantics of arithmetic commands

As one can notice from the table above, we do not have type-specific commands for **ADD**, **SUB** and **MUL** as these operations are closed under $\mathbb{Z}$ ¹. However, since this is not the case for division, we have separate instructions for integer and float division, namely **iDIV** and **fDIV**.

A similar approach will be followed for the boolean expressions in the next section.

¹Although syntactically different in SelVeri high level language; semantically, we do not interpret any integer differently than its real number counterpart.

2.2.3 Small-step operational semantics of boolean commands

$\langle \text{EQ} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 = v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{LE} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 \leq v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{LT} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 < v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{GE} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 \geq v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{GT} : c, v_1 : v_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, (v_1 > v_2) : e, \sigma, s, pc + 1 \rangle$ if $v_1, v_2 \in \mathbb{R}$
$\langle \text{AND} : c, b_1 : b_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 1 \wedge b_2 = 1 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 0 \vee b_2 = 0 \end{cases}$
$\langle \text{OR} : c, b_1 : b_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 1 \vee b_2 = 1 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = 0 \wedge b_2 = 0 \end{cases}$
$\langle \text{XOR} : c, b_1 : b_2 : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 \neq b_2 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b_1 = b_2 \end{cases}$
$\langle \text{NEG} : c, b : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, 1 : e, \sigma, s, pc + 1 \rangle & \text{if } b = 0 \\ \langle c, 0 : e, \sigma, s, pc + 1 \rangle & \text{if } b = 1 \end{cases}$

Table 3: Operational semantics of boolean commands

2.2.4 Small-step operational semantics of control-flow commands

$\langle \text{JZ } z : c, b : e, \sigma, s, pc \rangle$	$\triangleright$	$\begin{cases} \langle c, e, \sigma, s, pc + 1 \rangle & \text{if } b = 0 \\ \langle P[\mathcal{I}(z) :], e, \sigma, s, \mathcal{I}(z) \rangle & \text{if } b = 1 \wedge z \in \mathbf{Imm} \cap \mathbf{Int} \end{cases}$
$\langle \text{GOTO } z : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle P[\mathcal{I}(z) :], e, \sigma, s, \mathcal{I}(z) \rangle$ if $z \in \mathbf{Imm} \cap \mathbf{Int}$
$\langle \text{NOOP} : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, e, \sigma, s, pc + 1 \rangle$
$\langle i : c, e, \sigma, s, pc \rangle$	$\triangleright$	$\langle c, e, \hat{\sigma}, s, pc + 1 \rangle$ where i is any command

Table 4: Operational semantics of control-flow commands

The reader should note that any configuration which is not explicitly stated in the tables above lead to an error state, $\hat{\sigma}$ without any other change to the original state σ .

3 Compilation

Let **Code** and **CodeIR** be the sets of all **SelVeri** and **SelVerIR** programs, respectively. Using these sets, we will inductively define a compiler function:

$$\mathcal{C} : \mathbf{Code} \rightarrow \mathbf{CodeIR}$$

3.1 Compilation of arithmetic expressions

In order to formally define the compilation of arithmetic expressions, we define a helper function:

$$\mathcal{CA} : \mathbf{AExp} \rightarrow \mathbf{CodeIR}$$

$$\mathcal{CA}(x) = \text{PUSH } x \quad \text{if } x \in \mathbf{Imm} \cap (\mathbf{Int} \cup \mathbf{Float})$$

$$\mathcal{CA}(x) = \text{STORE } x \quad \text{if } x \in \mathbf{Var}$$

$$\mathcal{CA}(a_1 + a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{ADD}$$

$$\mathcal{CA}(a_1 * a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{MUL}$$

$$\mathcal{CA}(a_1 - a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{SUB}$$

$$\mathcal{CA}(a_1 / a_2) = \begin{cases} \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{iDIV} & \text{if } (a_1/a_2) \text{ has no subterm of type}^2 \\ & \mathbf{Float} \\ \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \text{fDIV} & \text{if } (a_1/a_2) \text{ has at least one subterm} \\ & \text{of type } \mathbf{Float} \end{cases}$$

²The types of subterms are determined by the compiler during the compile time.

3.2 Compilation of boolean expressions

In order to formally define the compilation of boolean expressions, we define a helper function:

$$\mathcal{CB} : \mathbf{BExp} \rightarrow \mathbf{CodeIR}$$

$$\mathcal{CB}(\mathbf{true}) = \mathbf{PUSH\ 1}$$

$$\mathcal{CB}(\mathbf{false}) = \mathbf{PUSH\ 0}$$

$$\mathcal{CB}(a) = \mathcal{CA}(a) \quad \text{if } a \in \mathbf{AExp}$$

$$\mathcal{CB}(a_1 = a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \mathbf{EQ}$$

$$\mathcal{CB}(a_1 \leq a_2) = \mathcal{CA}(a_2) : \mathcal{CA}(a_1) : \mathbf{LE}$$

$$\mathcal{CB}(\mathbf{not } b) = \mathcal{CB}(b) : \mathbf{NEG}$$

$$\mathcal{CB}(b_1 \mathbf{ and } b_2) = \mathcal{CB}(b_2) : \mathcal{CB}(b_1) : \mathbf{AND}$$

3.3 Compilation of statements

Finally, with the help of $\mathcal{CA}$ and $\mathcal{CB}$, we are ready to properly define the compilation function:

$$\mathcal{C} : \mathbf{Code} \rightarrow \mathbf{CodeIR}$$

$$\begin{aligned} \mathcal{C}(x : t) &= \begin{cases} \text{DECL INT } x & \text{if } t = \mathbf{Int} \\ \text{DECL FLOAT } x & \text{if } t = \mathbf{Float} \\ \text{DECL t } x : \text{DECL t_0 } x[0] : \dots : \text{DECL t_0 } x[n-1] & \text{if } \exists t_0 \in \mathbf{Type} \cdot \exists n \in \mathbb{N}^+ \cdot t = \mathbf{List}[t_0, n] \end{cases} \\ \mathcal{C}(x := a) &= \begin{cases} \mathcal{CA}(a) : \text{iSTORE } x & \text{if type}^4 \text{ of } x \text{ is } \mathbf{Int}. \\ \mathcal{CA}(a) : \text{fSTORE } x & \text{if type of } x \text{ is } \mathbf{Float}. \end{cases} \\ \mathcal{C}(\text{noop}) &= \text{NOOP} \\ \mathcal{C}(S_1; S_2) &= \mathcal{C}(S_1) : \mathcal{C}(S_2) \\ \mathcal{C}(\text{if } b \text{ then } S \text{ fi}) &= \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{NOOP} \\ \mathcal{C}(\text{if } b \text{ then } S_1 \text{ else } S_2 \text{ fi}) &= \mathcal{CB}(b) : \text{JZ } pc_{S_2} : \mathcal{C}(S_1) : \text{GOTO } pc_{\text{NOOP}} : \mathcal{C}(S_2) : \text{NOOP} \\ \mathcal{C}(\text{while } b \text{ do } S \text{ od}) &= \mathcal{CB}(b) : \text{JZ } pc_{\text{NOOP}} : \mathcal{C}(S) : \text{GOTO } pc_b : \text{NOOP} \end{aligned}$$

Please note that, during the compilation phase, compiler labels each compiled command with a corresponding program counter, using a simple incrementing counter. Therefore, pc variables seen in the compilation of the control-flow statements are actually replaced with syntactic integer-immediates.

⁶The type of variables are determined by the compiler during the compile time, with the help of scope.

4 Provably Correct Implementation

Now that the compilation function is explicitly defined, we need to prove the correctness of our implementation; by showing that given any **SelVeri** program P , we have

$$\langle P, \tilde{\sigma}, \tilde{s} \rangle \rightarrow^* \underline{\sigma}_f \quad \Longleftrightarrow \quad \langle \mathcal{C}(P), \epsilon, \tilde{\sigma}, \tilde{s}, 0 \rangle \triangleright^* \underline{\sigma}_f$$